Práctica 2: Pruebas de carga, Locust y Prometheus

Evaluación de Sistemas Informáticos

Grupo 16]

4 de mayo de 2026

Índice

1. Introducción y Objetivos
2. Monitorización y Enfoque EDR
3. Instrumentación del Código
4. Pruebas de Carga con Locust

Introducción y Objetivos

Objetivos principales de la práctica:

- Instrumentación del backend con métricas Prometheus].
- Despliegue y configuración de Prometheus y Grafana].
- Pruebas de carga y estrés con Locust].

Sistema Bajo Prueba:

- Servidor de autenticación OAuth2 desarrollado en Python (Flask y SQLite)[cite: 8].
- Implementa flujos clave del RFC 6749: *Authorization Code*, *Client Credentials* y *Refresh Token*[cite: 8].
- Ejecutado mediante el gestor `uv` en el puerto 5001].

Arquitectura de Monitorización

Implementación del modelo *pull* para la recopilación de métricas]:

App OAuth2 (5001) $\rightarrow$ **Prometheus (9090)** $\rightarrow$ **Grafana (3000)**[cite: 8]

Enfoque EDR (Endpoint Detection and Response):

- **Rendimiento:** Histogramas para evaluar tiempos de respuesta (umbral de 250ms) y control de peticiones simultáneas.
- **Estado de Tokens:** Monitorización de tokens activos, revocados, códigos pendientes y expirados.
- **Seguridad:** Detección de ataques de fuerza bruta (M2M y Web) e intentos de *Token Replay*.

Modificaciones en el Backend

La instrumentación se ha integrado siguiendo el principio DRY:

- `metrics.py`: Centralización de métricas (Counters, Gauges, Histograms) para evitar importaciones circulares. Uso estratégico de etiquetas (ej. `grant_type`)
- `app.py`: Exposición del endpoint `/metrics` vía `DispatcherMiddleware` y registro automático del tráfico HTTP con decoradores de Flask].
- `models.py`: Auditoría de la base de datos (códigos generados/consumidos, detección de tokens revocados).
- `oauth2_routes.py`: Seguimiento del ciclo de vida de los flujos de autenticación (iniciados, exitosos, fallidos).

Diseño de Escenarios en Locust

El `locustfile.py` simula un entorno realista mediante probabilidades: **50 % tráfico legítimo, 50 % tráfico malicioso**.

Tarea	Tipo	Peso	Prob.
`valid_client_credentials`	Tráfico legítimo M2M	4	40 %
`brute_force_api`	Ataque fuerza bruta M2M	2	20 %
`brute_force_user_login`	Ataque fuerza bruta web	2	20 %
`valid_user_login`	Tráfico legítimo web	1	10 %
`replay_attack`	Ataque token replay	1	10 %

Ejecución y Verificación

Configuración de Pruebas:

- **Carga normal:** 50 usuarios, *spawn rate* 5/s durante 5 min].
- **Prueba de estrés:** 200 usuarios, *spawn rate* 10/s durante 3 min].

Garantías del sistema:

- Limitación de almacenamiento en Prometheus (`retention.size=500MB`) para proteger los recursos de la máquina virtual.
- Uso de script de verificación (`test_metrics.py`) para validar manualmente los 9 escenarios de métricas antes del ataque global.

¡Gracias por su atención!

¿Preguntas?